KapasSetu AI: Optical Cotton Assay, Geo-Clustered Mandi & Edge Lint Inspection Suite

Tagline: *Bridging Farm-Gate Kapas to Clean Mill Bales via Calibrated Vision & Collective Market Power*
Origin: Upgraded Commercial Architecture from the *Nagpur RISE 2026 (Team Code Slayers)* Proposal
Build Philosophy: 100% Software-First Execution Today → Plug-and-Play Hardware Upgradation Tomorrow

---

1. Product Identity: What We Are Building & For Whom

1.1 Product Name: **KapasSetu AI**

- **Kapas:** Raw unginned seed cotton harvested by Indian farmers.
- **Setu:** Bridge—representing the digital trust and logistics bridge between isolated smallholder cotton farmers, Farmer Producer Organizations (FPOs), and industrial ginning mills.

1.2 What Exactly Are We Building?

We are building a **three-sided, 100% software-driven B2B2C agricultural quality and trade platform** with a built-in **Hardware Abstraction Layer (HAL)**:

1. **Calibrated Smartphone Cotton Grader (For Farmers):** A guided two-photo computer vision engine that uses everyday physical reference objects (a ₹5 coin and a white paper slip) plus local weather telemetry to measure **Color Grade (**$Rd, +\ b$**)**, **Trash %**, **Staple Length (**`$\text{mm}$`**)**, and **Estimated Moisture %** on any budget Android phone—without requiring any external hardware sensors.
2. **Geo-Clustered Virtual Mandi & Lot Aggregator (For Farmers, FPOs & Ginners):** A spatial backend powered by PostgreSQL/PostGIS that automatically groups small graded cotton batches (`$2\text{--}5\text{ quintals}$`) within a `$5\text{ km}$` village radius into commercial truckload lots (`$30\text{--}50\text{ quintals}$`) and broadcasts them to nearby ginning mills for direct farm-gate pickup bidding.
3. **Real-Time Lint Contamination & Virtual PLC Console (For Ginning Mills):** A local-first, `$<35\text{ ms}$` latency YOLOv8/v11 ONNX inspection dashboard that

detects white/colored plastic, polypropylene twine, hair, jute, and cloth in ginned lint streams at 30 FPS, triggers instant browser WebAudio sirens, and fires standardized HAL Webhooks to a **Software Virtual Conveyor Simulator** (which seamlessly hooks into physical 24V PLC relays in Phase 2).

1.3 Target Users & Value Proposition

User Persona	Role in Ecosystem	What They Use	Core Problem Solved	Direct Financial Impact
Smallholder Cotton Farmer *(Vidarbha & Pan-India)*	Producer ($1\text{--}5\text{ acres}$)	**KapasSetu Kisan PWA** *(100% Free, Offline-Ready, Marathi/Hindi/English)*	Eliminates arbitrary trader downgrades and connects small harvests directly to bulk mill buyers.	Reclaims **₹500–₹1,000 per quintal** ($15\%\text{--}20\%$ income increase).
FPO / Village Coordinator *(Krishi Mitra / Samiti)*	Village-Level Aggregator	**KapasSetu Pool Manager** *(Mobile/Web View)*	Automates quality verification and pools $10\text{--}20$ farmers into a single uniform truckload.	Earns transparent facilitation fee; eliminates quality disputes at collection.
Cotton Ginning Mill *(300+ Mills in Vidarbha)*	B2B Processor & Buyer	**KapasSetu Mill Console** *(₹4,500/mo SaaS per line)*	Prevents bale rejections from plastic/hair contamination and sources pre-graded, low-trash village lots directly.	Saves **₹27,000–₹54,000+ annually** in rejected bales + cuts middleman procurement commission.

---

2. Complete Problem Statement (PS)

2.1 The Core Problem Statement

> *"India's 60 million smallholder cotton farmers—led by $500,000+$ farmers in Maharashtra's Vidarbha belt—lose $15\%\text{--}20\%$ of their crop value (₹500–₹1,000 per quintal) at the farm gate due to subjective visual grading and middleman monopoly, while downstream ginning mills suffer a 10% bale rejection rate (losing ₹5,000–₹10,000 per rejected bale) due to undetected plastic, twine, and hair contamination during processing. Existing centralized High Volume Instrument (HVI) laboratory testing takes $2\text{--}7\text{ days}$, and naive single-photo smartphone AI apps fail in the field due to uncalibrated sunlight, invisible fiber staple length in raw piles, unaccounted moisture deductions, and the farmer's lack of alternative buyers."*

2.2 The Five Structural Root Causes We Must Solve

1. **The Optical Physics Bottleneck at the Farm Gate:** A raw pile of *Kapas* has fibers tightly clumped around seeds, making millimeter staple length ($24\text{ mm}\text{ vs. }30\text{ mm}$) invisible in a standard pile photo. Furthermore, harsh midday sunlight vs. shade shifts camera RGB values dramatically, corrupting color grade predictions.
2. **The Moisture Exploitation Loophole:** Even when cotton has bright color and low leaf trash, local traders arbitrarily claim moisture content exceeds the 8% benchmark (claiming $12\%\text{--}16\%$ moisture) to slash ₹400–₹700/quintal.
3. **The "Information Without Leverage" Trap:** Showing a price recommendation screen to a local village trader who holds informal credit leverage over a farmer with only 3 quintals of cotton does not create bargaining power. Farmers need **aggregated buyer access**, not just price tips.
4. **Gin Mill Speed vs. Cloud/SMS Latency:** Lint travels rapidly through ginning chutes. A 2-second cloud API inference paired with an SMS alert arrives after the contaminant is already pressed into a bale.
5. **Hardware Capital Friction:** Requiring farmers to buy special hardware attachments or requiring gin mills to overhaul PLC wiring before seeing software ROI stalls adoption.

---

3. Updated End-to-End Solution Methodology (The 4 Core Fixes)

Fix 1: Calibrated Dual-Photo Computer Vision + Agronomic Moisture Proxy

Instead of relying on a single uncalibrated photo, **KapasSetu AI** implements a deterministic, physics-backed **3-Step Guided Capture Protocol** in software:

1. **Photo 1 — Radiometric & Spatial Calibration on the Bulk Pile (Color $Rd/\ +\ b$ & Trash %):**
    - **User Action:** The farmer places a standard **₹5 coin** (exact diameter $= 23.0\text{ mm}$) on a **plain white paper slip** ($5\text{ cm} \times 5\text{ cm}$) resting on top of the raw cotton pile and captures a photo using the on-screen SVG target guide.
    - **Algorithmic White Balance:** OpenCV detects the white paper contour (`cv2.findContours`), measures the RGB deviation from pure white $(255, 255, 255)$, applies a Von Kries chromatic adaptation transform, and converts the image to the device-independent **CIE** $L^*a^*b^*$ color space.
    - **HVI-Equivalent Color & Trash Calculation:**
        - **Reflectance (Rd):** Mapped directly from the calibrated luminance channel L^*.

    - **Yellowness (+ b):** Mapped from the calibrated b^* channel.
    - **Trash Percentage (`$T_{\%}$`):** Adaptive Gaussian thresholding isolates dark leaf, bark, and bract particles against the bright lint surface, calculating exact pixel area ratio: `$$T_{\%} = \left( \frac{N_{\text{trash\_pixels}}}{N_{\text{cotton\_pixels}}} \right) \times 100$$`
2. **Photo 2 — Combed Seed-Halo Radial Measurement (Millimeter Staple Length):**
    - **User Action:** The farmer pulls a single cotton seed from the pile, fans the fibers outward into a flat circular "halo" (the standard manual hand-stapling prep), and places it on a dark cloth/surface next to the **₹5 coin**.
    - **Pixel-to-Millimeter Calibration:** Hough Circle Transform (`cv2.HoughCircles`) detects the ₹5 coin diameter in pixels (`$D_{\text{coin\_px}}$`), establishing the exact spatial resolution factor: `$$R_{\text{mm/px}} = \frac{23.0\text{mm}}{D_{\text{coin\_px}}}$$`
    - **Radial Fiber Measurement:** Otsu thresholding segments the dark inner seed core (radius `$r_{\text{seed}}$`) and the outer fibrous halo boundary. Using polar coordinate projection from the seed centroid across 360°, the engine computes the `$75\text{th}$`-percentile radial fiber extension (r_{75}) and outputs true staple length (`$L_{\text{staple}}$`) in millimeters: `$$L_{\text{staple}} = \left( r_{75} - r_{\text{seed}} \right) \times R_{\text{mm/px}}$$`
3. **Step 3 — Software-First Agronomic Moisture Proxy (With Future BLE Sensor Hook):**
    - **Software Calculation Now:** The farmer taps two visual toggles (*Picking Time: Morning Dew vs. Afternoon Sun* and *Days Dried in Shade: `$0, 1\text{--}2, \text{ or } 3+$`*). The backend combines this with real-time GPS Relative Humidity (`$RH_{\%}$`) and temperature from the **Open-Meteo API** to compute an empirical equilibrium moisture content (`$M_{\text{est}}$`) between `$7.5\%$` and `$15.0\%$`.
    - **Hardware Upgradation Hook:** The database schema includes `moisture_source: ENUM('SOFTWARE_PROXY', 'BLE_CAPACITIVE_PROBE')`, allowing a Bluetooth moisture probe to overwrite `$M_{\text{est}}$` in Phase 2 without changing API contracts.

Fix 2: Geo-Clustered Village Lot Aggregation & Realistic 2026–27 MSP Pricing Engine

To give farmers real bargaining power, **KapasSetu AI** converts individual grades into collective commercial lots:

1. **Algorithmic Fair Price Valuation (P_{fair}):**
    - Uses realistic Indian market benchmarks anchored to the **Government of India Minimum Support Price (MSP)** and live **eNAM / Agmarknet APMC** modal prices ($P_{\text{base}} \approx \text{₹7,121 to ₹8,667 per quintal}$ depending on Medium vs. Long Staple class).
    - Calculates net fair farm-gate price per quintal: $$P_{\text{fair}} = P_{\text{base}} + M_{\text{staple}} - D_{\text{trash}} - D_{\text{moisture}}$$ Where:
        - M_{staple}: Staple premium ($+\text{₹150/qtl}$ per mm above 27.5 mm; $-\text{₹120/qtl}$ per mm below 26.0 mm).
        - D_{trash}: Trash penalty (₹0 if $T_{\%} \le 3.0\%$; else $(T_{\%} - 3.0) \times 0.01 \times P_{\text{base}}$).
        - D_{moisture}: Moisture penalty (₹0 if $M_{\text{est}} \le 8.0\%$; else $(M_{\text{est}} - 8.0) \times 0.012 \times P_{\text{base}}$).
2. **PostGIS 5 km Village Lot Clustering:**
    - Once graded, the farmer enters their harvest quantity (e.g., 4.5 quintals) and taps **"Join Village Pool for Direct Mill Pickup"**.
    - A PostgreSQL + PostGIS spatial query (`ST_DWithin`) groups all uncommitted farmer lots within a $5,000\text{-meter}$ radius that belong to the same quality tier (*Tier A: Long Staple* $\ge 28\text{ mm}$, *Tier B: Medium-Long* $26\text{--}27.9\text{ mm}$, *Tier C: Medium* $<26\text{ mm}$).
    - When the pooled weight crosses the **Commercial Truckload Threshold ($\ge 30.0\text{ quintals}$)**, the status transitions to `READY_FOR_BIDDING`, and all subscribed ginning mills within 50 km receive a real-time lot notification.
    - Ginners bid directly on the pooled lot via their console; farmers accept the highest bid via one-tap SMS/PWA confirmation, bypassing the exploitative local trader completely.

Fix 3: Sub-35ms Local Edge-Browser / FastAPI Lint Inspection + Hardware Abstraction Layer (HAL)

To catch fast-moving contaminants on a ginning chute without buying industrial edge hardware on Day 1:

1. **Zero-Cloud Local Inference Pipeline (30 FPS, $<35\text{ ms}$ Latency):**

    - Runs **YOLOv8n** (or **YOLOv11n**) quantized to **ONNX Runtime** locally on the gin mill's existing office laptop/PC connected to a standard USB webcam or RTSP IP camera mounted above the lint slide.
    - Detects 5 critical contaminant classes: `white_pp_plastic`, `colored_plastic`, `jute_twine`, `human_hair_clump`, and `cloth_rag`.
2. **Instant Multi-Modal Operator Alerting:**
    - Instead of slow Twilio SMS messages, any detection with confidence ≥ 0.70 triggers an immediate **WebAudio API synthesized industrial siren** (`$880\text{ Hz}$` square-wave alarm) on the PC speakers, flashes a full-screen red strobe overlay, and caches the freeze-frame bounding box for one-click operator logging.
3. **Hardware Abstraction Layer (HAL) & Virtual PLC Simulator:**
    - Every detection fires a standardized JSON webhook to a local HAL service (`POST /api/v1/hal/actuator-trigger`) over HTTP/WebSockets.
    - **Software Build Now:** The Ginner Console renders a live **Virtual Conveyor & Air-Jet Diverter Simulator** that visually halts the animated conveyor belt for `$5\text{ seconds}$` and logs the relay state transition (`RELAY_OPEN` → `CONVEYOR_PAUSED`).
    - **Future Hardware Upgrade Later:** In Phase 2, an ESP32 microcontroller or NVIDIA Jetson Orin Nano listens to that exact same webhook/MQTT topic and trips a physical `$24\text{V}$` industrial relay or pneumatic air-jet valve with **zero backend or frontend code changes**.

Fix 4: Sustainable B2B SaaS + Marketplace Business Model

- **100% Free Forever for Farmers:** No subscriptions, no ads, and zero hardware purchase required for farmers.
- **Gin Mill B2B SaaS Subscription: ₹4,500 / month per ginning line** during the 6-month ginning season (includes unlimited local ONNX contamination detection, quality compliance logs for spinning mills, and direct access to pre-graded village cotton pools).
- **Direct Procurement Success Fee: 0.5% platform clearing fee** paid by the winning ginning mill on aggregated village lot transactions (still saving the ginner `$2\%\text{--}4\%$` compared to traditional commission agents).

---

4. Complete System Architecture & Monorepo Structure

```
None
kapassetu-ai/
├── docker-compose.yml                # Orchestrates PostGIS, Redis, Backend, ML
Worker, Frontends
├── README.md
├── backend/                          # FastAPI Core + Marketplace + HAL Webhook
Bus
│   ├── app/
│   │   ├── main.py                   # FastAPI entry point & CORS/WebSocket
setup
│   │   ├── config.py                 # Env vars, MSP constants, thresholds
│   │   ├── database.py               # SQLAlchemy + GeoAlchemy2 (PostGIS)
connection
│   │   ├── models.py                 # DB Tables: Farmers, Assays,
VillagePools, Bids, HalEvents
│   │   ├── schemas.py                # Pydantic v2 request/response validation
│   │   ├── services/
│   │   │   ├── cv_grading.py         # OpenCV Coin/White-Card CIE L*a*b* & Halo
Staple Engine
│   │   │   ├── moisture_proxy.py     # Open-Meteo API + Agronomic Moisture
Estimator
│   │   │   ├── pricing_engine.py     # 2026-27 MSP + eNAM Fair Price Calculator
│   │   │   ├── lot_aggregator.py     # PostGIS 5km spatial clustering & 30-qtl
pool trigger
│   │   │   ├── lint_inspector.py     # YOLOv8n ONNX Runtime local frame
processor
│   │   │   └── hal_controller.py     # Hardware Abstraction Layer (Virtual PLC
+ Webhook dispatcher)
│   │   └── routers/
│   │       ├── grading.py            # POST /api/v1/grade/analyze
│   │       ├── marketplace.py        # POST /api/v1/lots/publish, GET
/api/v1/lots/pools, POST /bid
│   │       └── ginner_hal.py         # WS /ws/ginner/stream, POST
/api/v1/hal/actuator-trigger
│   └── requirements.txt
├── frontend-kisan/                   # Farmer & FPO Mobile-First PWA (React 18
+ Vite + Tailwind)
│   ├── src/
│   │   ├── components/
│   │   │   ├── DualCameraCapture.jsx # Guided SVG overlay for Coin/White-Card &
Seed Halo
│   │   │   ├── MoistureChecklist.jsx # 2-tap picking time & drying input
│   │   │   ├── GradeCertificate.jsx  # Digital Assay Card + MSP Fair Price
Breakdown
│   │   │   └── VillagePoolCard.jsx   # Live progress bar (e.g., 24/30 Quintals
pooled in 5km)
```

```
|   |   └── services/offlineSync.js     # IndexedDB caching for low-connectivity
villages
└── frontend-ginner/                    # Gin Mill B2B Inspection & Lot Bidding
Console (React 18)
    ├── src/
    |   ├── components/
    |   |   ├── LiveLintScanner.jsx     # 30 FPS WebRTC/Canvas + Bounding Boxes +
WebAudio Siren
    |   |   ├── VirtualPlcHalWidget.jsx # Software Simulator for Conveyor Relay &
Air-Jet Valve
    |   |   ├── VillageLotMap.jsx       # Interactive map of 30+ quintal graded
pools & 1-click bidding
    |   |   └── ContaminationLog.jsx    # False-positive feedback toggle & bale QA
export
```

---

5. Step-by-Step Software Implementation Plan (4-Week Execution Blueprint)

Week 1: Core Backend, PostGIS Spatial Database & 2026–27 Pricing Engine

- **Goal:** Stand up the PostgreSQL + PostGIS database, FastAPI server, agronomic moisture proxy, and the MSP/eNAM pricing engine.
- **Deliverables:**
  - Working Docker container running `postgis/postgis:15-3.3`.
  - Automated pricing calculation matching official Medium Staple (₹7,121–₹8,267/qtl) and Long Staple (₹7,521–₹8,667/qtl) baselines with exact staple, trash, and moisture adjustments.
  - Spatial clustering logic (`ST_DWithin`) that automatically creates or joins a `VillagePool` within $5,000\text{ meters}$.

Week 2: Calibrated OpenCV Cotton Grading Engine (Coin + White Card + Halo)

- **Goal:** Build the deterministic computer vision module in `cv_grading.py` that eliminates lighting bias and extracts millimeter staple length without specialized hardware.
- **Deliverables:**
  - White-card contour detection and CIE $L^*a^*b^*$ white-balance normalization.

- Hough Circle ₹5 coin detector outputting exact $R_{\text{mm/px}} = 23.0 / D_{\text{px}}$.
- Seed-halo polar radial profiler outputting staple length in mm and adaptive trash segmentation outputting $T_{\%}$.

Week 3: Farmer PWA (`frontend-kisan`) with Guided Dual-Camera UI & Pool Joining

- **Goal:** Build the mobile-first React PWA with visual camera guides, regional language support (Marathi/Hindi/English), and one-tap village pool enrollment.
- **Deliverables:**
  - HTML5 `navigator.mediaDevices.getUserMedia` camera screen with SVG target overlays (a circular target for the ₹5 coin and a rectangular box for the white paper card).
  - Interactive Digital Quality Certificate displaying Color Grade, Staple Length, Trash %, Moisture %, Fair Price per Quintal, and nearby Village Pool progress bar.

Week 4: Ginner B2B Console (`frontend-ginner`), Local ONNX Scanner & Virtual HAL

- **Goal:** Build the real-time ginner contamination dashboard, WebAudio alarm synthesizer, Virtual Conveyor HAL simulator, and direct village lot bidding interface.
- **Deliverables:**
  - Local YOLOv8n ONNX inference pipeline streaming bounding boxes over WebSockets/Canvas at $\ge 25\text{--}30\text{ FPS}$.
  - Browser WebAudio API siren (880 Hz pulsing tone) + red strobe alert when contamination confidence ≥ 0.70.
  - Live **Virtual PLC Conveyor Widget** reacting to `POST /api/v1/hal/actuator-trigger` events, demonstrating hardware readiness for Phase 2.

---

6. Copy-Paste Master Prompts for End-to-End Code Generation

You can copy and paste the following **5 Master Prompts** sequential-style into an AI coding IDE (Cursor, Windsurf, Claude Code, or ChatGPT) to generate the complete, production-ready codebase module by module.

MASTER PROMPT 1: FastAPI Backend Core, PostGIS Schema, Pricing Engine & Spatial Lot Aggregator

None

```
Act as a Principal AgriTech Backend Architect. Build the complete production-ready
Python FastAPI backend (`backend/`) for "KapasSetu AI" — an optical cotton
grading, geo-clustered village lot aggregation, and gin-mill contamination
inspection platform.

### REQUIREMENTS & TECH STACK
1. Framework: Python 3.11+, FastAPI, SQLAlchemy 2.0 (async or sync), GeoAlchemy2
(PostGIS), Pydantic v2, Uvicorn, Requests/HTTPX.
2. Database: PostgreSQL with PostGIS extension (`GEOGRAPHY(Point, 4326)`), plus an
automatic SQLite + Haversine fallback mode (`USE_SQLITE_FALLBACK=True` in `.env`)
so a developer can run the entire backend locally in 10 seconds even before
starting Docker PostGIS.

### DATABASE MODELS (`app/models.py`)
Create complete SQLAlchemy models for:
1. `Farmer`: id, phone, name, village_name, district (default "Yavatmal"),
latitude, longitude, created_at.
2. `CottonAssay`: id, farmer_id, reflectance_rd (float), yellowness_plus_b
(float), color_grade (str: "Suvin/Super Fine", "Grade A - Good Middling", "Grade B
- Middling", "Grade C - Low Middling"), staple_length_mm (float), trash_percent
(float), moisture_percent (float), moisture_source (str: "SOFTWARE_PROXY" or
"BLE_SENSOR"), pixel_to_mm_ratio (float), base_msp_price (float),
staple_adjustment (float), trash_deduction (float), moisture_deduction (float),
final_fair_price_per_qtl (float), quantity_quintals (float), latitude (float),
longitude (float), pool_id (FK to VillagePool, nullable), created_at.
3. `VillagePool`: id, center_lat (float), center_lon (float), village_cluster_name
(str), quality_tier (str: "TIER_A_LONG", "TIER_B_MEDIUM_LONG", "TIER_C_MEDIUM"),
total_quintals (float), target_threshold_quintals (float, default 30.0),
avg_staple_mm (float), avg_trash_pct (float), avg_moisture_pct (float),
ask_price_per_qtl (float), status (str: "COLLECTING", "READY_FOR_BIDDING",
"LOCKED_BY_MILL"), created_at.
4. `MillBid`: id, pool_id (FK), gin_mill_name (str), gin_mill_location (str),
bid_price_per_qtl (float), pickup_date (str), status (str: "PENDING", "ACCEPTED",
"REJECTED"), created_at.
5. `HalActuatorEvent`: id, mill_id (str), camera_line_id (str), contaminant_class
(str), confidence (float), action_command (str: "STOP_CONVEYOR_AND_PULSE_AIRJET"),
relay_duration_ms (int, default 5000), hardware_mode (str: "VIRTUAL_SIMULATOR" or
"PHYSICAL_PLC_GPIO"), timestamp.

### BUSINESS LOGIC TO IMPLEMENT
1. `app/services/moisture_proxy.py`:
   - Function `estimate_cotton_moisture(lat: float, lon: float, picking_session:
str, shade_dried_days: int, sensor_override_pct: float | None = None) -> dict`
   - If `sensor_override_pct` is provided, return it with source="BLE_SENSOR".
```

```
   - Otherwise query Open-Meteo API
(`https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=re
lative_humidity_2m,temperature_2m`) with a 2-second timeout and fallback default
(RH=55%, Temp=32C).
   - Calculate empirical cotton moisture: base 8.5% + ((RH - 50) * 0.06) + (2.2%
if picking_session == "MORNING_DEW" else 0.0%) - (min(shade_dried_days, 4) *
0.65%). Clamp between 7.0% and 16.0%.

2. `app/services/pricing_engine.py`:
   - Function `calculate_fair_kapas_price(staple_mm: float, trash_pct: float,
moisture_pct: float, color_grade: str) -> dict`
   - Use realistic Indian cotton MSP & Mandi baselines:
     - Base price (`P_base`): Rs 7,521/quintal if `staple_mm >= 27.5` (Long Staple
benchmark), else Rs 7,121/quintal (Medium Staple benchmark).
     - Staple adjustment (`M_staple`): `(staple_mm - 27.5) * 150.0` if `staple_mm
>= 27.5`, else `(staple_mm - 26.0) * 120.0`.
     - Color grade adjustment: +Rs 200 if "Grade A", Rs 0 if "Grade B", -Rs 250 if
"Grade C".
     - Trash deduction (`D_trash`): Rs 0 if `trash_pct <= 3.0`, else `(trash_pct -
3.0) * 0.012 * P_base`.
     - Moisture deduction (`D_moisture`): Rs 0 if `moisture_pct <= 8.0`, else
`(moisture_pct - 8.0) * 0.015 * P_base`.
     - Return all individual components, `final_fair_price_per_qtl` rounded to 2
decimals, and a farmer negotiation tip string.

3. `app/services/lot_aggregator.py`:
   - Function `assign_assay_to_village_pool(db, assay: CottonAssay) ->
VillagePool`
   - Determine quality tier: "TIER_A_LONG" if `staple_mm >= 28.0`,
"TIER_B_MEDIUM_LONG" if `26.0 <= staple_mm < 28.0`, else "TIER_C_MEDIUM".
   - Search for an existing `VillagePool` with `status == "COLLECTING"` and
matching `quality_tier` within 5.0 km of `(assay.latitude, assay.longitude)`
(using PostGIS `ST_DWithin` or Haversine distance formula in SQLite fallback
mode).
   - If found, add `assay.quantity_quintals` to `pool.total_quintals`, update
weighted averages for staple, trash, moisture, and ask_price. If
`pool.total_quintals >= pool.target_threshold_quintals` (30.0 quintals),
automatically flip `pool.status = "READY_FOR_BIDDING"`.
   - If not found, create a new `VillagePool`.

### API ENDPOINTS (`app/main.py` & routers)
Expose clean REST endpoints with full CORS enabled:
- `POST /api/v1/grade/analyze` (Multipart form or base64 images + metadata -> runs
CV grading, moisture proxy, pricing engine, and optionally joins a pool).
- `POST /api/v1/lots/join-pool` (Assigns an existing assay to a 5km VillagePool).
- `GET /api/v1/lots/pools` (Returns all active VillagePools with farmer count,
progress %, and bids).
```

```
- `POST /api/v1/lots/pools/{pool_id}/bid` (Allows a gin mill to place a direct pickup bid on a pool).
- `POST /api/v1/lots/bids/{bid_id}/accept` (Locks the pool and accepts the gin mill's bid).
- `POST /api/v1/hal/actuator-trigger` (Logs a lint contamination event and broadcasts a HAL relay command to connected WebSockets).

Provide complete, runnable code for every file with zero placeholders.
```

MASTER PROMPT 2: Calibrated OpenCV Cotton Grading Engine (Coin + White Card + Seed Halo)

None

```
Act as a Senior Computer Vision Engineer specializing in Agricultural Commodity Grading. Write the complete, self-contained Python module `backend/app/services/cv_grading.py` using OpenCV (`cv2`) and NumPy (`np`) for "KapasSetu AI".

### OBJECTIVE
Solve the two biggest physical failures of smartphone cotton grading without external hardware:
1. Eliminate ambient lighting bias using a white paper reference card in Photo 1 (Bulk Cotton Pile) to extract calibrated CIE L*a*b* Reflectance (Rd), Yellowness (+b), and Trash %.
2. Measure true millimeter fiber Staple Length in Photo 2 (Combed Seed Halo on a dark background) using a standard Indian Rs 5 coin (exact physical diameter = 23.0 mm) as a spatial scale reference.

### FUNCTIONS TO IMPLEMENT IN `cv_grading.py`

1. `calibrate_white_balance_and_extract_pile_metrics(pile_img_bgr: np.ndarray) -> dict`:
   - Detect the brightest neutral rectangular contour in the image representing the white reference card (using HSV thresholding for low saturation, high value, and `cv2.findContours`).
   - If the white card is detected, compute the mean RGB values `(R_w, G_w, B_w)` inside the card region and apply a diagonal Von Kries white balance scaling (`255.0 / mean_channel`) across the entire image. If no card is found (e.g., during quick demo testing), apply Gray-World / CLAHE illumination normalization and set `white_card_detected = False`.
   - Mask out the white reference card and convert the cotton pile region to CIE L*a*b* (`cv2.cvtColor(balanced_bgr, cv2.COLOR_BGR2LAB)`).
   - Compute:
```

```
     - `reflectance_rd`: Scaled from mean L* channel into standard HVI Rd range
(55.0 to 85.0).
     - `yellowness_plus_b`: Scaled from mean b* channel into standard HVI +b range
(6.0 to 16.0).
     - `trash_percent`: Convert the cotton ROI to grayscale, apply
`cv2.adaptiveThreshold` combined with dark intensity masking to isolate dark leaf,
bract, and bark specks, and calculate `(trash_pixels / total_cotton_pixels) *
100.0`, clamped to realistic range (0.5% to 12.0%).
     - `color_grade`: Classify based on `reflectance_rd` and `yellowness_plus_b`
into:
       - `"Grade A - Good Middling (HVI Rd >= 75)"`
       - `"Grade B - Middling (HVI Rd 68-74)"`
       - `"Grade C - Spotted / Low Middling (HVI Rd < 68)"`

2. `measure_seed_halo_staple_length(halo_img_bgr: np.ndarray) -> dict`:
   - Detect the Rs 5 coin (23.0 mm diameter) using `cv2.HoughCircles` on a
Gaussian-blurred grayscale image (with contour circularity fallback if
HoughCircles misses).
   - Calculate `pixel_to_mm_ratio = 23.0 / coin_diameter_px` (fallback to a
calibrated default based on image width if no coin is present in synthetic test
images, setting `coin_detected = False`).
   - Mask out the coin region and segment the bright cotton fiber halo against the
dark background using Otsu thresholding (`cv2.threshold(..., cv2.THRESH_BINARY +
cv2.THRESH_OTSU)`) and morphological closing.
   - Locate the dark central cotton seed core centroid `(cx, cy)` and estimate the
inner seed radius `r_seed_px`.
   - Sample 72 radial rays (every 5 degrees from 0 to 360) outward from `(cx, cy)`
to the outer boundary of the segmented fiber halo mask.
   - Compute the 75th percentile radial fiber distance `r_75_px` (equivalent to
HVI Upper Half Mean Length - UHML).
   - Calculate `staple_length_mm = (r_75_px - r_seed_px) * pixel_to_mm_ratio`,
clamped to biological cotton bounds (21.0 mm to 36.0 mm).

3. `generate_annotated_preview_base64(pile_img_bgr, halo_img_bgr, metrics) ->
dict`:
   - Draw clean green/cyan OpenCV overlays showing the detected white card,
detected trash specks, detected Rs 5 coin (23mm scale), and radial staple
measurement circle, returning base64-encoded JPEG strings so the farmer and judges
can visually verify the AI's math on screen.

Write clean, well-documented, defensive OpenCV code that never crashes on noisy
smartphone photos.
```

MASTER PROMPT 3: Real-Time Lint Contamination Engine & Hardware Abstraction Layer (HAL) Simulator

None

```
Act as an Industrial Edge AI & Embedded Systems Architect. Build the real-time
gin-mill lint contamination detection service
(`backend/app/services/lint_inspector.py` and
`backend/app/services/hal_controller.py`) for "KapasSetu AI".

### OBJECTIVE
Replace slow 2-second cloud inference and Twilio SMS alerts with a sub-35ms local
inspection pipeline that processes video frames at 30 FPS, detects critical cotton
lint contaminants, triggers immediate browser audio/visual alarms, and dispatches
standardized Hardware Abstraction Layer (HAL) commands to a Virtual PLC Conveyor
Simulator (ready for physical ESP32 / Jetson Nano 24V relay connection in Phase
2).

### TECHNICAL SPECIFICATIONS
1. `lint_inspector.py`:
   - Support loading an ONNX Runtime (`onnxruntime`) YOLOv8n/v11n model
(`models/cotton_contaminant_yolov8n.onnx`) for the 5 classes:
     - `0: white_pp_plastic` (White polypropylene fertilizer/raffia bag shred)
     - `1: colored_plastic` (Colored polyethylene wrapper)
     - `2: jute_twine` (Gunny bag jute string)
     - `3: human_hair_clump` (Hair strand bundle)
     - `4: cloth_rag` (Colored cotton/synthetic rag piece)
   - Include an automatic **High-Speed Color/Texture Anomaly Fallback Detector**
(using OpenCV HSV saturation/hue contrast + white specular plastic sheen +
morphological filament analysis) when the `.onnx` weight file is not yet present
on disk, so the live webcam/video demo works out-of-the-box at 30+ FPS (<15ms
latency) on any laptop.
   - Output per frame: `inference_latency_ms`, `fps`, `contaminants_detected`
(list of `{class_name, confidence, bbox: [x1, y1, x2, y2], severity: "CRITICAL" |
"WARNING"}`), and `trigger_hal_relay: bool` (True if any detection has `confidence
>= 0.70`).

2. `hal_controller.py` (Hardware Abstraction Layer):
   - Maintain in-memory + DB state of the ginning line actuator:
     - `conveyor_state`: `"RUNNING"` or `"PAUSED_BY_AI"`
     - `air_jet_diverter_state`: `"IDLE"` or `"FIRING_VALVE"`
     - `strobe_siren_state`: `"SILENT"` or `"ACTIVE_ALARM"`
     - `connected_hardware_driver`: `"SOFTWARE_VIRTUAL_PLC_V1"` (with configurable
`EXTERNAL_PLC_WEBHOOK_URL` for Phase 2 ESP32/Modbus-TCP relay).
   - When `trigger_actuator(mill_id, camera_id, detections)` is called:
     - Immediately transition `conveyor_state = "PAUSED_BY_AI"`,
`air_jet_diverter_state = "FIRING_VALVE"`, and `strobe_siren_state =
"ACTIVE_ALARM"`.
     - Emit a structured JSON HAL payload:
```

```
      ```json
      {
        "event_id": "HAL-2026-XXXX",
        "protocol": "KAPASSETU_HAL_V1",
        "gpio_pin_target": "RELAY_CH1_24V_NC",
        "air_valve_pin": "SOLENOID_CH2_8BAR",
        "command": "HALT_FEEDER_AND_EJECT",
        "hold_duration_ms": 5000,
        "contaminant": "white_pp_plastic",
        "confidence": 0.91,
        "inference_latency_ms": 18.4
      }
      ```
    - Provide a reset endpoint `POST /api/v1/hal/reset-line` (and auto-reset timer after 5 seconds) so the operator can mark the contaminant as removed or flag a false positive.

Write complete, modular Python code and integrate it with FastAPI endpoints in `app/routers/ginner_hal.py`.
```

MASTER PROMPT 4: Farmer Mobile PWA (`frontend-kisan`) with Guided Dual-Camera UI & Pool Joining

None

```
Act as a Principal Frontend Engineer. Build the complete mobile-first Farmer Progressive Web App (`frontend-kisan/`) for "KapasSetu AI" using React 18, Vite, Tailwind CSS, and Lucide React icons.

### DESIGN & UX REQUIREMENTS
- Built for Vidarbha cotton farmers and FPO village coordinators using budget Android smartphones in bright outdoor lighting.
- High-contrast outdoor UI (Emerald green, warm amber, crisp white cards, large 48px+ tap targets).
- Instant language switcher at the top: **English | मराठी (Marathi) | हिन्दी (Hindi)** with complete translation dictionaries for all farmer-facing labels.

### CORE SCREENS & WORKFLOW TO IMPLEMENT
1. **Guided 2-Step Optical Capture Wizard (`DualCameraCapture.jsx`)**:
   - **Step 1: Bulk Pile + White Card Capture**:
     - Opens live smartphone camera (`navigator.mediaDevices.getUserMedia({ video: { facingMode: "environment" } })`) OR allows uploading/selecting built-in realistic sample images for instant judge testing.
```

```
     - Renders an on-screen SVG target overlay showing a dashed rectangle labeled
*"Place White Paper Slip Here (Auto Lighting Calibration)"* over the raw cotton
pile.
   - **Step 2: Combed Seed Halo + Rs 5 Coin Capture**:
     - Renders an on-screen SVG circular target on the left labeled *"Place ₹5
Coin (23mm Scale)"* and a larger radial target on the right labeled *"Place Combed
Single Seed Halo on Dark Cloth"*.
   - **Step 3: 2-Tap Moisture & Lot Weight Input (`MoistureChecklist.jsx`)**:
     - Toggle 1: Picking Time (*Morning Dew (7–10 AM)* vs. *Sunny Afternoon (11
AM–5 PM)*).
     - Toggle 2: Shade Drying (*Freshly Picked Today*, *1–2 Days Dried*, *3+ Days
Dried*).
     - Harvest Quantity Slider/Input: `1.0` to `25.0 Quintals` (Default `4.0
Quintals`) + Village Selector (*Yavatmal - Kalamb*, *Wardha - Hinganghat*,
*Amravati - Dhamangaon*, *Nagpur - Katol*).

2. **Digital Cotton Assay Certificate & Fair Price Screen
(`GradeCertificate.jsx`)**:
   - Displays the OpenCV calibrated visual overlays side-by-side.
   - Displays the 4 core quality metrics with HVI benchmark badges:
     - **Color Grade:** e.g., `Grade A - Good Middling` (`Rd: 76.4`, `+b: 8.9`)
     - **Staple Length:** e.g., `28.8 mm (Long Staple)` with `Scale: ₹5 Coin (23.0
mm)` badge
     - **Trash Content:** e.g., `2.4% (Low Leaf/Bark)`
     - **Moisture Content:** e.g., `8.2% (Agronomic Weather Proxy - RH 52%)` with
a subtle pill showing `"BLE Moisture Probe: Not Connected (Using Software
Proxy)"`.
   - **Transparent 2026–27 MSP & Mandi Fair Price Breakdown Card**:
     - Shows Base MSP/Mandi Price (e.g., `₹7,521/qtl`), Staple Premium
(`+₹195/qtl`), Trash Deduction (`₹0`), Moisture Deduction (`-₹23/qtl`), and **Net
Fair Farm-Gate Price (`₹7,693 / quintal`)**.
     - Shows expected total batch value and a bold negotiation banner: *"Do not
sell below ₹7,550/qtl to local traders! Pool your lot below to get direct Gin Mill
bids."*

3. **5km Geo-Clustered Village Pool & Direct Mill Bidding
(`VillagePoolCard.jsx`)**:
   - One-tap button: **"Add My 4.0 Quintals to Village Pool (5 km Radius)"**.
   - Live progress bar showing the village pool filling up toward the `30.0
Quintal` commercial truckload threshold (e.g., `28.5 / 30.0 Quintals Pooled across
7 Farmers in Kalamb Cluster`).
   - Once the pool crosses `30.0 Quintals`, display live incoming direct pickup
bids from nearby Gin Mills (e.g., *Mahalaxmi Ginning Pressing, Hinganghat – Bid:
₹7,740/qtl + Free Farm-Gate Tractor Pickup*).

Provide complete, production-ready React components with clean state management
and seamless API integration.
```

MASTER PROMPT 5: Ginner B2B Console (`frontend-ginner`) with 30 FPS Lint Scanner, WebAudio Siren, Virtual PLC HAL & Lot Bidding

None

```
Act as a Principal Industrial IoT & B2B SaaS Frontend Architect. Build the
complete Ginner Inspection & Procurement Console (`frontend-ginner/`) for
"KapasSetu AI" using React 18, Vite, Tailwind CSS, and Lucide React.

### CORE MODULES TO BUILD

1. **Real-Time 30 FPS Lint Contamination Scanner (`LiveLintScanner.jsx`)**:
   - Supports connecting to a live USB/RTSP camera feed (`getUserMedia`) OR
running an interactive **High-Speed Ginning Chute Simulation Stream** where moving
cotton lint travels down a chute and the user (or automated timer) can inject
realistic contaminants (*White PP Fertilizer Bag Strip*, *Colored Plastic
Wrapper*, *Jute Twine*, *Human Hair Clump*) to test real-time detection and HAL
interlocking.
   - Renders real-time bounding boxes (`[x1, y1, x2, y2]`), class labels,
confidence scores, and live telemetry badges (`Latency: 18.2 ms`, `Frame Rate: 30
FPS`, `Engine: YOLOv8n-ONNX Local`).
   - **WebAudio API Industrial Siren**: When a contaminant with confidence >= 70%
is detected, immediately synthesize a loud, pulsing 880Hz/660Hz industrial klaxon
siren using the browser's native `AudioContext` (no external MP3 files required)
and flash a red strobe border around the viewport.

2. **Hardware Abstraction Layer (HAL) & Virtual PLC Relay Simulator
(`VirtualPlcHalWidget.jsx`)**:
   - Displays a real-time schematic of the ginning line hardware state driven by
the backend HAL webhook (`/api/v1/hal/actuator-trigger`):
     - **Feeder Conveyor Motor (Relay CH1 - 24V NC):** Shows animated moving belt
(`RUNNING` in green -> instantly flips to `HALTED BY AI` in red when contamination
is detected).
     - **Pneumatic Air-Jet Diverter (Solenoid CH2 - 8 Bar):** Flashes `EJECTING
CONTAMINANT` with a 5-second countdown timer.
     - **Live HAL JSON Telemetry Terminal:** Displays the exact raw JSON/MQTT
payload emitted by the software (`{"protocol": "KAPASSETU_HAL_V1",
"gpio_pin_target": "RELAY_CH1_24V_NC", "command": "HALT_FEEDER_AND_EJECT"}`) and a
toggle switch showing **"Current Mode: Software Virtual PLC Simulator | Phase 2
Ready: ESP32 / Jetson Orin GPIO"**.
   - Includes Operator Action buttons: **"Confirm Contaminant Removed & Resume
Line"** and **"Flag False Positive (Retrain Model)"**.

3. **Direct Village Cotton Pool Marketplace (`VillageLotMap.jsx`)**:
   - Displays all active 5km geo-clustered village cotton pools in Vidarbha
(`COLLECTING` and `READY_FOR_BIDDING` lots >= 30 Quintals).
```

```
   - Shows verified aggregate quality metrics for each village pool (Total
Quintals, Number of Farmers, Weighted Avg Staple Length in mm, Avg Trash %, Avg
Moisture %, and Ask Price/qtl).
   - Allows the gin mill manager to submit a **Direct Pickup Bid** (Price per
quintal + Pickup Date) with an automatic ROI calculator showing how much the mill
saves in middleman commission and trash/contamination loss.

Provide complete, self-contained, visually stunning dark-mode industrial React
code.
```

---

7. Verification & Pitch Readiness Checklist (How to Demo This to Judges & Investors)

1. **Prove Fix 1 (Optical Physics Solved):** Place a real ₹5 coin and a small piece of white paper next to a cotton sample during your live demo. Show how the app draws a green circle around the 23 mm coin and a box around the white card, proving on-screen how lighting and millimeter scale are mathematically calibrated.
2. **Prove Fix 2 (Farmer Leverage Solved):** Submit a 6-quintal graded batch on the Farmer PWA and watch the **Kalamb Village Pool** cross the 30-quintal threshold, immediately triggering a live bid notification on the Ginner Console at realistic 2026–27 prices (₹7,500–₹7,800/quintal).
3. **Prove Fix 3 (Gin Mill Speed & Hardware Upgradability Solved):** Trigger a white polypropylene plastic detection on the Ginner Console. Within $<35\text{ ms}$, the WebAudio siren sounds, the **Virtual PLC Conveyor Belt** halts on screen, and the live HAL Webhook JSON payload is displayed—proving the software is 100% complete today and ready for a 24V relay wire tomorrow.
4. **Prove Fix 4 (Commercial Viability Solved):** Highlight the B2B SaaS unit economics (₹4,500/month per ginning line + 0.5% direct lot clearing fee), keeping the farmer app 100% free forever.